

Методы оптимизации в машинном обучении

Лекция № 2

Ю.Т. Каганов к.т.н., доцент

Кафедра ИУ-5

МГТУ им. Н.Э. Баумана

План

- 1. Оптимизация с учетом ограничений.**
- 2. Задачи условной оптимизации.**
- 3. Теорема Каруша-Куна-Таккера**
 - ККТ условия
 - Двойственность в выпуклой оптимизации
 - Регуляризация (L1, L2, Dropout)
- 4. SVM - Машины опорных векторов**

Оптимизация с учетом ограничений

1. Задачи выпуклого программирования и теорема Куна-Таккера.
2. Двойственные задачи.
3. Методы штрафных функций.
4. Пример: использование теоремы Куна-Таккера в методе опорных векторов.

Методы условной оптимизации

(методы математического программирования с учетом ограничений)

Методы нелинейного программирования.

1. **Прямые методы** – это методы, где последовательно решаются задачи оптимизации таким образом, что на каждом этапе процесс решения не выходит за пределы активных ограничений.
 - 1.1. Методы проекции градиента.
 - 1.2. Методы решения уравнений Куна-Таккера.
2. **Двойственные методы** – ряд методов, которые строят вспомогательную функцию по аналогии с двойственной функцией Лагранжа.
 - 2.1. **Методы штрафных функций**
 - 2.1.2. Методы внешних штрафов.
 - 2.1.3. Методы внутренних штрафов (метод барьерных функций).
 - 2.1.4. Метод последовательной безусловной оптимизации (МПБМ-SUMT).
 - 2.2. **Лагранжевы методы.**
 - 2.2.1. Метод Удзавы и Эрроу-Гурвица.
 - 2.2.2. Методы модифицированных функций Лагранжа.

Необходимые и достаточные условия существования экстремума функции

Условия существования экстремума с учетом ограничений.

Постановка задачи:

Пусть $f: X \rightarrow Y; X \subset E^n, Y \subset E^1$ $y=f(x)$ – выпуклая функция;

$x \in X, y \in Y; f(x) \in C^1(X)$ и пусть $g: X \rightarrow Z; X \subset E^n, Z \subset E^m$
 $f(x) \in C^1(X)$

$g_i(x) \in C^1(X), j = 1..m.$ – вогнутые или выпуклые функции,

$g = [g_1(x), g_2(x), \dots, g_m(x)]^T$ а также $\forall g_j(x) = 0$ – ограничения типа равенства.

Тогда задачу поиска экстремума функции при наличии ограничений можно записать:

$$\begin{cases} x^e = \text{Arg min } f(x) \\ x_i \geq 0, i = 1..n \\ g_j(x) = 0, j = 1..m \end{cases}$$

Необходимые и достаточные условия существования экстремума функции.

Метод множителей Лагранжа.

(1788г."Аналитическая механика")

Функция вида: $L(x, \lambda) = f(x) + \sum_{j=1}^m \lambda_j g_j(x).$

Если взять вариацию от функции $L(x, \lambda)$ в $U_\varepsilon(x^e)$:

$$\delta L(x, \lambda) = \sum_{i=1}^n \frac{\partial f(x)}{\partial x_i} \delta x_i + \sum_{i=1}^n \sum_{j=1}^m \lambda_j \frac{\partial g_j}{\partial x_i} \delta x_i = 0 \quad \text{при} \quad x = x^e$$

$$\delta L(x, \lambda) = \sum_{i=1}^n \left[\frac{\partial f}{\partial x_i} + \sum_{j=1}^m \lambda_j \frac{\partial g_j}{\partial x_i} \right] \delta x_i = 0$$

Таким образом получаем систему уравнений : $\left[\frac{\partial f}{\partial x_i} + \sum_{j=1}^m \lambda_j \frac{\partial g_j}{\partial x_i} = 0 \right]_{x=x^e}$

Подбором определенных значений λ_j можно добиться того, чтобы эти уравнения удовлетворялись при $\forall i = 1..n$. Именно в точке x^e достигается стационарность функции $L(x, \lambda)$, т.е. экстремум функции $f(x)$ при заданных ограничениях $g_j(x)=0$.

ЗАДАЧА НЕЛИНЕЙНОГО ПРОГРАММИРОВАНИЯ

$$f(x) \rightarrow \min;$$

$$h_j(x) = 0, j = 1, 2, \dots, l; \quad \text{Или}$$

$$g_k(x) \leq 0, k = l + 1, \dots, m.$$

$$f(x) \rightarrow \min;$$

$$h(x) = 0,$$

$$g(x) \leq 0.$$

$$L(x, \lambda, \mu) = \lambda_0 f(x) + \sum_{j=1}^l \lambda_j h_j(x) + \sum_{k=l+1}^m \mu_k g_k(x)$$

Элементы X , удовлетворяющие ограничениям задачи называются **допустимыми решениями**.

Терема Куна-Таккера (Каруша-Джона)

Если точка x^* есть решение задачи математического программирования, то найдутся такие числа $\lambda_0^*, \lambda_1^*, \lambda_2^*, \dots, \lambda_l^*$, и $\mu_{l+1}^*, \dots, \mu_m^*$ не все равные нулю, что $\lambda_0^* \nabla f(x^*) + \sum_{j=1}^l \lambda_j^* \nabla h_j(x^*) + \sum_{k=l+1}^m \mu_k^* \nabla g_k(x^*) = 0$, причем $\lambda_0^* \geq 0$ и все $\mu_k^* \geq 0$, при этом $\mu_k^* g_k(x^*) = 0, k = l + 1, \dots, m$ - называется условием **дополняющей нежесткости**.

Двойственность в задачах ОПТИМИЗАЦИИ

- В контексте линейного программирования, каждая задача (прямая) имеет соответствующую двойственную задачу. Если у нас есть задача минимизации:
- $\min c^T x$
- при условии $Ax \leq b, x \geq 0$,
- то соответствующая двойственная задача будет выглядеть следующим образом:
- $\max b^T y$
- при условии $A^T y = c, y \geq 0$.
- Здесь:
- c — вектор коэффициентов целевой функции,
- A — матрица ограничений,
- b — вектор правых частей ограничений,
- x — вектор переменных прямой задачи,
- y — вектор двойственных переменных (множителей Лагранжа $y = \lambda$)

Двойственность в машинном обучении

- В машинном обучении концепция двойственности часто используется в контексте задач, связанных с методом опорных векторов (SVM). В SVM мы стремимся максимизировать зазор между классами, что может быть сформулировано как задача оптимизации:
- $\min \frac{1}{2} \|w\|^2$
- При условии $y_i(w^T x_i + b) \geq 1, \forall i,$
- где w — вектор весов, b — смещение, x_i — входные данные, а
- y_i — метки классов.
- Соответствующая двойственная задача будет:
- $\max \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^m \alpha_i \alpha_j y_i y_j (x_i^T x_j)$
- при условии: $\sum_{i=1}^n \alpha_i y_i = 0, \alpha_i \geq 0.$

Теорема Куна-Таккера в машинном обучении

- **Теорема Куна-Таккера** (Kuhn-Tucker) — это фундаментальный результат в теории математической оптимизации, обобщающий метод множителей Лагранжа на задачи с ограничениями в форме неравенств. Она играет ключевую роль в машинном обучении, особенно в задачах обучения с ограничениями, SVM (Support Vector Machines) и других оптимизационных моделях.

Формулировка теоремы

Рассмотрим задачу оптимизации:

- $\min_x f(x)$
- при условиях: $g_i(x) \leq 0, i = 1, \dots, m, \quad h_j(x) = 0, j = 1, \dots, p,$
- где $f(x), g_i(x), h_j(x)$ — дифференцируемые функции.
- 1. Стационарность: $\nabla f(x^*) + \sum_{i=1}^m \lambda_i \nabla g_i(x^*) + \sum_{j=1}^p \mu_j \nabla h_j(x^*) = 0.$
- 2. Допустимость: $g_i(x^*) \leq 0, h_j(x^*) = 0.$
- 3. Условия дополняющей нежёсткости: $\lambda_i g_i(x^*) = 0, \lambda_i \geq 0.$
- 4. Неотрицательность множителей: $\lambda_i \geq 0.$

Здесь λ_i и μ_j — множители Лагранжа для неравенств и равенств соответственно.

- **Достаточные условия:** Если $f(x)$ выпукла, $g_i(x)$ выпуклы, $h_j(x)$ аффинны, а точка x^* удовлетворяет условиям Куна-Таккера, то x^* — глобальный минимум.

ЗАДАЧА НЕЛИНЕЙНОГО ПРОГРАММИРОВАНИЯ

$$f(x) \rightarrow \min;$$

$$h_j(x) = 0, j = 1, 2, \dots, l; \quad \text{Или}$$

$$g_k(x) \leq 0, k = l + 1, \dots, m.$$

$$f(x) \rightarrow \min;$$

$$h(x) = 0,$$

$$g(x) \leq 0.$$

$$L(x, \lambda, \mu) = \lambda_0 f(x) + \sum_{j=1}^l \lambda_j h_j(x) + \sum_{k=l+1}^m \mu_k g_k(x)$$

Элементы X , удовлетворяющие ограничениям задачи называются **допустимыми решениями**.

Терема Куна-Таккера (Каруша-Джона)

Если точка x^* есть решение задачи математического программирования, то найдутся такие числа $\lambda_0^*, \lambda_1^*, \lambda_2^*, \dots, \lambda_l^*$, и $\mu_{l+1}^*, \dots, \mu_m^*$ не все равные нулю, что $\lambda_0^* \nabla f(x^*) + \sum_{j=1}^l \lambda_j^* \nabla h_j(x^*) + \sum_{k=l+1}^m \mu_k^* \nabla g_k(x^*) = 0$, причем $\lambda_0^* \geq 0$ и все $\mu_k^* \geq 0$, при этом $\mu_k^* g_k(x^*) = 0, k = l + 1, \dots, m$ - называется условием **дополняющей нежесткости**.

Двойственность в задачах оптимизации и в машинном обучении.

Двойственность — это важный концепт как в теории оптимизации, так и в машинном обучении. Она позволяет получить дополнительную информацию о проблемах оптимизации и может значительно упростить вычисления.

Двойственность в задачах ОПТИМИЗАЦИИ

- В контексте линейного программирования, каждая задача (прямая) имеет соответствующую двойственную задачу. Если у нас есть задача минимизации:
- $\min c^T x$
- при условии $Ax \leq b, x \geq 0$,
- то соответствующая двойственная задача будет выглядеть следующим образом:
- $\max b^T y$
- при условии $A^T y = c, y \geq 0$.
- Здесь:
- c — вектор коэффициентов целевой функции,
- A — матрица ограничений,
- b — вектор правых частей ограничений,
- x — вектор переменных прямой задачи,
- y — вектор двойственных переменных (множителей Лагранжа $y = \lambda$)

Двойственность в машинном обучении

- В машинном обучении концепция двойственности часто используется в контексте задач, связанных с методом опорных векторов (SVM). В SVM мы стремимся максимизировать зазор между классами, что может быть сформулировано как задача оптимизации:
- $\min \frac{1}{2} \|w\|^2$
- При условии $y_i(w^T x_i + b) \geq 1, \forall i,$
- где w — вектор весов, b — смещение, x_i — входные данные, а
- y_i — метки классов.
- Соответствующая двойственная задача будет:
- $\max \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^m \alpha_i \alpha_j y_i y_j (x_i^T x_j)$
- при условии: $\sum_{i=1}^n \alpha_i y_i = 0, \alpha_i \geq 0.$

Двойственность в машинном обучении

Двойственность в задачах оптимизации и машинном обучении предоставляет мощные инструменты для анализа и решения задач. Она позволяет не только находить оптимальные решения, но и понимать взаимосвязи между различными задачами.

Метод опорных векторов (SVM)

- **1. Основные формулы и алгоритм**
- **Линейный SVM для бинарной классификации**
- **Цель:** Найти гиперплоскость, максимально разделяющую классы.
- **Уравнение разделяющей гиперплоскости:**
- $w^T x + b = 0$
- где w - вектор нормали, b - смещение
- **Классификация точки:**
- $y = \text{sign}(w^T x + b)$
- **Задача оптимизации**
- **Прямая задача (Hard Margin):**
- $\min\{w, b\} \left(\frac{1}{2}\right) ||w||^2$
-
- при условиях: $y_i(w^T x_i + b) \geq 1$, для всех i

Метод опорных векторов (SVM)

- **Soft Margin SVM** (с допустимыми ошибками):
- $\min\{w, b\} \left(\frac{1}{2} \right) \|w\|^2 + C \sum \xi_i$
-
- при условиях:
- $-y_i(w^T x_i + b) \geq 1 - \xi_i$
- $\xi_i \geq 0$
- где:
- **C** - параметр регуляризации (баланс между максимизацией зазора и минимизацией ошибок)
- ξ_i - переменные слабины (slack variables)

Метод опорных векторов (SVM)

- 2. "Ядерный трюк" (Kernel Trick)
- Концепция
- Вместо явного отображения данных в высокоразмерное пространство $\varphi(x)$, используем **функцию ядра**:
 - $K(x_i, x_j) = \varphi(x_i)^T \varphi(x_j)$
 - Двойственная задача с ядром
 - $\max_{\alpha} \sum \alpha_i - \left(\frac{1}{2}\right) \sum \sum \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
 - Классификация:
 - $f(x) = \text{sign}(\sum \alpha_i y_i K(x_i, x) + b)$

Метод опорных векторов (SVM)

- Популярные функции ядра
- Линейное ядро:
 - $K(x, x') = x^T x'$
- Полиномиальное ядро:
 - $K(x, x') = (\gamma x^T x' + r)^d$
- **RBF** (Радиальная базисная функция, Гауссово ядро):
 - $K(x, x') = \exp(-\gamma \|x - x'\|^2)$
- Сигмоидное ядро:
 - $K(x, x') = \tanh(\gamma x^T x' + r)$

Метод опорных векторов (SVM)

- **Преимущества ядерного трюка**
 - . Не нужно явно вычислять $\phi(x)$
 - . Работа в бесконечномерных пространствах
 - . Вычислительная эффективность

Метод опорных векторов (SVM)

3. Алгоритм обучения *SVM*

- Шаги:

1. Выбор ядра и параметров (C, γ и др.)
2. Формирование задачи оптимизации с матрицей ядер
3. Решение задачи квадратичного программирования:
 1. *Sequential Minimal Optimization (SMO)*
 2. *Gradient descent methods*
 3. *Coordinate descent*
4. Определение опорных векторов ($\alpha_i > 0$)
5. Вычисление смещения b :

$$b = y_s - \sum \alpha_i y_i K(x_i, x_s)$$

для опорного вектора x_s

Метод опорных векторов (SVM)

- **Двойственная задача (Dual Problem)**
- $\max_{\alpha} \sum \alpha_i - \left(\frac{1}{2}\right) \sum \sum \alpha_i \alpha_j y_i y_j (x_i^T x_j)$
-
- при условиях:
- $- 0 \leq \alpha_i \leq C$
- $-\sum \alpha_i y_i = 0$
- **Решение:**
- $w = \sum \alpha_i y_i x_i$
- **Опорные векторы** - объекты с $\alpha_i > 0$

SVM vs Нейронные сети

Аспект	SVM	Нейронные сети
-----	-----	-----
Оптимизация	Выпуклая (QP)	Невыпуклая
Решение	Глобальный оптимум	Локальный минимум
Гиперпараметры	C, kernel params	Много (архитектура, η , ...)
Интерпретируемость	Опорные векторы	Черный ящик
Масштабируемость	Плохая ($O(n^2)$)	Хорошая (SGD)
Feature engineering	Важен выбор ядра	Автоматический
Малые данные	✓ Хорошо	⚠ Переобучение
Большие данные	✗ Проблемы	✓ Отлично
Высокая размерность	✓ Kernel trick	✓ Deep learning

Когда использовать SVM:

- ✓ Малые/средние датасеты (< 10K примеров)
- ✓ Нужны теоретические гарантии
- ✓ Важна интерпретируемость
- ✓ Табличные данные с хорошими признаками

Когда использовать нейросети:

- ✓ Большие данные (> 100K примеров)
- ✓ Сложные структурированные данные (изображения, текст, аудио)
- ✓ Нужна автоматическая feature extraction
- ✓ Есть вычислительные ресурсы (GPU)

МЕТОДЫ ОПТИМИЗАЦИИ В МАШИННОМ ОБУЧЕНИИ

1. Goodfellow I., Bengio Y., Courville A. "Deep Learning"

MIT Press, 2016. **Библия глубокого обучения.** Глава 8: Optimization for Training Deep Models. Охватывает SGD, Adam, momentum, batch normalization. Доступна бесплатно: www.deeplearningbook.org.

2. Bottou L., Curtis F.E., Nocedal J. "Optimization Methods for Large-Scale Machine Learning"

SIAM Review, 2018, Vol. 60, No. 2. **Обзорная статья (100+ страниц).** Stochastic gradient methods, mini-batch methods. От классики до современных подходов Обязательно к прочтению!

3. Ruder S. "An Overview of Gradient Descent Optimization Algorithms"

arXiv:1609.04747, 2016. Популярный обзор алгоритмов градиентного спуска. SGD, Momentum, Nesterov, Adagrad, RMSprop, Adam. Практичный и понятный Доступен бесплатно на arxiv.org.

4. Sutskever I. "Training Recurrent Neural Networks"

PhD Thesis, University of Toronto, 2013. Оптимизация для RNN Hessian-free optimization. Влиятельная работа от сооснователя OpenAI.

5. Zhang A., Lipton Z.C., Li M., Smola A.J. "Dive into Deep Learning"

Cambridge University Press, 2023. **Современный интерактивный учебник.** Практическая оптимизация для DL. Код на PyTorch/TensorFlow. Бесплатно: d2l.ai.

6. Kingma D.P., Ba J. "Adam: A Method for Stochastic Optimization"

ICLR, 2015 (arXiv:1412.6980). Оригинальная статья про Adam optimizer. Один из самых используемых алгоритмов. Обязательно для понимания современной практики.

Спасибо за внимание!

Каганов Юрий Тихонович

yurijkaganov@gmail.com

[kaganov y.t@bmstu.ru](mailto:kaganov.y.t@bmstu.ru)